Hanoi University of Science and Technology
School of Information and Communications Technology

Spectrogram-Based Machine Learning Approaches for Multi-Label Bird Sound Classification

PROJECT REPORT

Ninh Minh Hieu
Data Science & AI
HUST
Hieu.nm2416688@hust.sis.edu.vn

Hoang Bao Huy
Data Science & AI
HUST
Huy.hb16705@sis.hust.edu.vn

Nguyen Cong Hung
Data Science & AI
HUST
hung.nc2416702@sis.hust.edu.vn

Nguyen Hoang Quan
Data Science & AI
HUST
Quan.nh2416738@sis.hust.edu.vn

Le The Quyen
Data Science & AI
HUST
Quyen.lt2416743@sis.hust.edu.vn

Course: Machine Learning
Class ID: 168272
Semester: 20252
Supervisor: Dr. Dam Quang Tuan

Hanoi, 2026

Contents

1 Introduction

1.1 Dataset Background

Bioacoustic monitoring is an important tool for studying biodiversity, ecological change, and species distribution in natural environments. Instead of relying only on visual observation, passive acoustic monitoring records environmental soundscapes and allows researchers to detect species from their vocalizations. Bird sound recognition is a representative problem in this field because many bird species have distinctive calls, songs, chirps, trills, and frequency-modulated patterns.

In this project, we use the BirdCLEF 2025 dataset as the main benchmark for studying bird and wildlife sound classification. The task is to estimate the presence probability of each target class from audio recordings. This setting is challenging because real-world recordings are noisy, labels may be incomplete, multiple species can vocalize in the same clip, and the distribution of species is highly imbalanced. In addition, there is a domain gap between directed recordings, which often focus on a single target species, and passive soundscape recordings, which are longer and contain richer background noise.

From a machine learning perspective, this task is naturally formulated as a multi-label audio classification problem. Each audio segment may contain zero, one, or multiple target species. Therefore, the model must output independent probabilities for all classes rather than using a single softmax prediction. This motivates the use of spectrogram-based convolutional networks, sound event detection, BCE-based training losses, semi-supervised learning, and ensemble inference.

1.2 Project Objectives

Our group studies a complete spectrogram-centered pipeline for bird sound recognition. The main objectives are:

- To analyze the BirdCLEF dataset and identify the main modeling challenges, including class imbalance, multi-label annotations, domain shift, and unlabeled soundscape data;
- To build a spectrogram-based convolutional baseline using EfficientNet-family backbones;
- To extend the baseline into a sound event detection model that preserves temporal information and uses attention pooling;
- To investigate imbalance-aware training using sampling and controlled augmentation, while considering class weighting, focal loss, and label smoothing as optional strategies rather than separately validated components;
- To study semi-supervised approaches, including pseudo-labeling, Noisy Student training, and self-distillation;

- To evaluate ensemble-based inference strategies, including multi-checkpoint averaging, shifted-window inference, and temporal smoothing.

2 Dataset and Exploratory Analysis

2.1 Dataset Overview

We use the BirdCLEF 2025 dataset for our internal method study. The training set contains a total of 28,564 audio samples, corresponding to approximately 280.5 hours of audio recordings.

The test data consists of one-minute soundscape recordings collected through passive acoustic monitoring. All recordings are provided in Ogg format and resampled to 32 kHz. The prediction setting is to estimate the probability of presence for each of the 206 target classes in every five-second chunk of each audio file.

The training recordings come from three major sources: Xeno-Canto, iNaturalist, and the CSA collection provided by the Instituto Humboldt. Table 1 shows the number of samples from each source.

Table 1: Number of training samples by data source.

Source	**Number of samples**
Xeno-Canto	21,204
iNaturalist	7,198
Other/CSA	162

In addition to primary labels, the metadata also includes secondary labels that describe background species appearing in the recordings. These labels are important because the test soundscapes may contain multiple species at the same time. In our dataset analysis, secondary species information appears in 4,958 recordings, and the secondary-label field is available for 28,539 recordings, which is approximately 99.9% of the training data. This information can be used to better model the multi-label structure of the test set.

2.2 Challenges

2.2.1 Class Imbalance

The dataset has severe class imbalance, which is expected in ecological audio data because some species are naturally much rarer than others and therefore have fewer available recordings. In our dataset, the most common species is `grekis` with 990 samples, while the rarest species, `64862`, has only 2 samples. This gives an imbalance ratio of approximately 495×. There are 39 species with fewer than 10 samples, while only 9 species have more than 500 samples.

This imbalance is a major modeling problem. If the model is trained naively, it may learn to prioritize frequent species and ignore rare species. Therefore, we need to address class imbalance

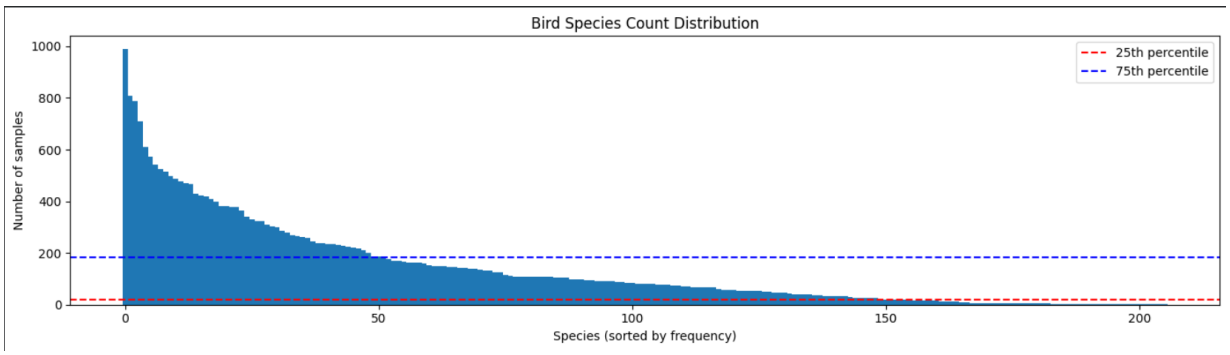


Figure 1: Distribution of animal classes in the BirdCLEF 2025 training dataset.

using techniques such as class-balanced sampling, loss reweighting, data augmentation, and careful validation.

2.2.2 Multi-Label Nature

One recording may contain multiple bird species simultaneously, such as crow, robin, and sparrow. The presence of secondary labels confirms that many recordings contain background species in addition to the primary target species. This means the problem should be treated as multi-label classification rather than single-label classification.

2.2.3 Domain Shift Between Recordings

There is a significant domain shift between passive acoustic monitoring soundscapes and directed recordings from sources such as Xeno-Canto. Directed recordings often focus on a target bird and may have clearer vocalizations, while soundscapes are longer, noisier, and contain more overlapping environmental sounds. However, some species that are underrepresented in the labeled training dataset may appear frequently in soundscape data. This creates an additional challenge, but it also motivates the use of semi-supervised learning and pseudo-labeling.

2.2.4 Unlabeled Data

Unlabeled soundscape data provides an opportunity for pseudo-labeling and other semi-supervised learning techniques. If used carefully, these recordings can reduce data loss and help the model adapt to the test-domain soundscape distribution.

3 Data Preprocessing

3.1 Audio Segmentation

Audio segmentation is an important preprocessing step because the BirdCLEF prediction setting requires predictions at a fixed temporal resolution. The dataset contains recordings with different durations: many soundscape recordings are close to one minute long, while some directed recordings are shorter than five seconds. Since our model is designed to predict bird

presence on five-second audio segments, all input audio must be converted into fixed-length chunks before spectrogram generation and model training.

All recordings are first resampled to a sampling rate of 32,000 Hz. Therefore, one five-second segment contains

$$32{,}000 \times 5 = 160{,}000$$

raw audio samples. This fixed input length is useful because it ensures that every training example produces a spectrogram with a consistent time dimension. It also matches the prediction setting, where each one-minute soundscape is evaluated as a sequence of five-second chunks.

For audio recordings longer than five seconds, we use two sampling strategies during training. At each training iteration, one of the two strategies is selected randomly with equal probability. The first strategy chooses a random starting time from the interval

$$[0, L - 5],$$

where L is the audio duration in seconds. The segment from start to start$+5$ is then extracted. This random cropping strategy increases training variability because the model can see different parts of the same recording across epochs. It also helps the classifier become less dependent on a fixed temporal position of the bird call.

The second strategy simply selects the first five seconds of the recording. Although this may seem less diverse than random cropping, it is useful for directed bird recordings. In many field recordings, the recorder starts when the target animal is already vocalizing and stops when the vocalization ends. As a result, the beginning of the audio file often contains strong evidence for the primary label. Keeping the first five seconds allows the model to learn from this informative region instead of always relying on randomly selected segments that may contain weaker or noisier portions.

Combining these two strategies gives a balance between reliability and diversity. The first-five-second crop preserves the likely target vocalization in directed recordings, while the random crop acts as a data augmentation method that exposes the model to different background conditions, call positions, and possible secondary species. This is especially helpful for long soundscape files, where important species may appear only briefly within the full recording.

During validation, we use a deterministic segmentation rule to make the evaluation reproducible. Specifically, only the first five seconds of each validation recording are used to generate the prediction. This avoids randomness in validation scores and makes model comparisons fair across different experiments. For test-time soundscape inference, the same five-second resolution can be extended naturally by splitting each one-minute recording into consecutive non-overlapping five-second chunks.

For recordings shorter than five seconds, padding is required. If an audio track is shorter than four seconds, we pad it with silence or very low-amplitude empty noise until it reaches five seconds. Padding ensures that short clips can still be processed by the same spectrogram pipeline and neural network architecture. Using empty noise instead of repeating the signal

avoids artificially duplicating bird calls, which could distort the temporal structure of the original recording. After segmentation and padding, every input clip has exactly 160,000 waveform samples and can be converted into a fixed-size log-mel spectrogram.

3.2 Spectrogram Generation

In this project, each audio clip is converted into a log-mel spectrogram before being passed to the neural network. This subsection focuses on the preprocessing steps that transform raw waveform samples into a normalized time–frequency representation. The horizontal axis describes time, the vertical axis describes frequency, and each value represents the signal energy at a particular time and frequency region.

All audio recordings are first resampled to a common sampling rate of 32,000 Hz and converted to mono. Each training sample is represented by a fixed-duration audio window of five seconds. If the original recording is shorter than the required length, zero padding is applied; if it is longer, a segment is selected from the recording. During training, random cropping is used to expose the model to different parts of the same recording, while validation uses a deterministic crop to ensure reproducible evaluation.

For each audio window $x(t)$, we compute a short-time Fourier transform (STFT) using a window size of 2048 samples. The hop length is set to 768 samples, which controls the temporal resolution between adjacent frames. The power spectrogram is then projected onto the mel scale using 192 mel frequency bins. The frequency range is limited to approximately 50 Hz–15,000 Hz, because this range captures most useful bird vocalization patterns while reducing irrelevant very-low-frequency noise and near-Nyquist artifacts.

The mel-spectrogram is computed as

$$M = \mathrm{Mel}\left(|\,\mathrm{STFT}(x)|^2\right),$$

where $\mathrm{Mel}(\cdot)$ denotes the mel filter-bank projection. We then convert the power values into a logarithmic scale:

$$S_{\log} = 10\log_{10}(M + \epsilon),$$

where ϵ is a small constant used to avoid taking the logarithm of zero. The logarithmic representation is important because audio energy has a large dynamic range, and the log scale makes weak but meaningful bird calls easier for the model to learn.

Finally, each spectrogram is normalized by subtracting its mean and dividing by its standard deviation:

$$\hat{S} = \frac{S_{\log} - \mu}{\sigma + \epsilon}.$$

The resulting normalized log-mel spectrogram has the shape $1 \times F \times T$, where F is the number of mel bins and T is the number of time frames. This tensor shape is then used by the modeling components described in the Model Architecture section.

3.3 Handling Rare Species

Rare species are one of the main challenges in the dataset. As shown in the dataset analysis, the most common species has 990 recordings, while the rarest species has only 2 recordings. In total, 39 species have fewer than 10 samples. This creates a strong imbalance problem because the model receives many more training signals from frequent species than from rare species.

If the training data is used directly, the classifier may learn a biased decision boundary that favors common classes. As a result, the model can achieve a strong overall score while still failing to recognize underrepresented species. This is especially problematic in ecological audio analysis because rare species may be biologically important even if they appear in only a small number of recordings.

Rare species are also more difficult in the multi-label setting. They may appear as weak background sounds, secondary labels, or short acoustic events inside longer recordings. In addition, some rare species may occur more often in soundscape-style audio than in clean directed recordings. Therefore, rare-species recognition is affected by both limited labeled data and the domain shift between directed recordings and natural soundscapes.

Because of these challenges, the training strategy must expose the model to rare classes more effectively while avoiding severe overfitting to a small number of repeated recordings. The concrete strategies for addressing this issue, including class-aware sampling, controlled augmentation, and conservative pseudo-label usage, are discussed in Section 5.2.

4 Model Architecture

4.1 Spectrogram as Image Classification

In the baseline formulation, the normalized log-mel spectrogram from the preprocessing stage is treated as an image-like tensor. This does not repeat the spectrogram construction procedure; instead, it defines how the resulting representation is consumed by the neural model. For a spectrogram with F mel-frequency bins and T time frames, the input can be written as

$$\mathbf{X} \in \mathbb{R}^{C \times F \times T},$$

where C denotes the number of channels. For a single log-mel spectrogram, $C = 1$. If the spectrogram is repeated across channels or combined with additional acoustic channels, then C may be set to 3 for compatibility with pretrained image backbones.

A convolutional neural network can process this tensor because local time–frequency patterns in spectrograms behave similarly to local visual patterns in images. Convolutional filters can detect short chirps, harmonic bands, rising or falling frequency sweeps, and repeated syllable structures. Lower layers capture local acoustic edges or bands, while deeper layers combine them into more discriminative species-level features.

This representation allows bird audio classification to be mapped directly to a standard

multi-label image classification pipeline. A mini-batch of B spectrograms is written as

$$\mathbf{X}_{\text{batch}} \in \mathbb{R}^{B \times C \times F \times T},$$

and the CNN produces a prediction matrix

$$\hat{\mathbf{Y}} \in [0,1]^{B \times K},$$

where K is the number of target bird species. Since BirdCLEF recordings may contain multiple bird species, the target label is a multi-hot vector rather than a single categorical label. The final layer therefore uses independent sigmoid outputs instead of a softmax function:

$$\hat{y}_{i,k} = \sigma(z_{i,k}), \qquad k = 1, \ldots, K,$$

where $z_{i,k}$ is the logit for class k in sample i.

At the clip level, a training example corresponds to a short segment, for example a five-second or ten-second crop from a longer recording. The spectrogram of this crop is passed through the CNN and assigned the labels associated with the clip. At the frame level, the time dimension of the same spectrogram tensor may be preserved more explicitly, allowing the model to produce local predictions for different temporal regions. These frame-level scores can then be aggregated by max pooling, mean pooling, or attention pooling to obtain a clip-level prediction. Thus, the same spectrogram representation can support both simple clip-level image classification and more detailed sound event detection.

The baseline image-classification formulation has several practical advantages. First, it enables the use of EfficientNet as a mature image-based backbone. Second, it allows standard data augmentation techniques to be adapted to audio spectrograms, including time masking, frequency masking, random cropping, mixup, and cutmix-like operations. Third, it simplifies implementation because the training loop is almost identical to that of multi-label image classification. For these reasons, we use spectrogram-as-image classification as the starting point before adding more advanced components such as attention pooling, pseudo-labeling, and ensembling.

4.2 EfficientNet Backbone

In our model, the input audio is first converted into a log-mel spectrogram. A log-mel spectrogram can be treated as an image-like representation, where the horizontal axis represents time and the vertical axis represents frequency. Therefore, convolutional neural networks can be used to extract local time–frequency patterns from the spectrogram. In this setting, EfficientNet is used as a backbone feature extractor. Its role is to transform the input spectrogram into high-level acoustic features before the final prediction head produces species probabilities.

EfficientNet is a family of convolutional neural networks designed to achieve a strong balance between accuracy and computational efficiency. Traditional CNN scaling often increases only

one aspect of the model, such as depth, width, or input resolution. However, increasing only one dimension may lead to an inefficient architecture. A deeper network may learn more complex hierarchical features, but it may become harder to train. A wider network may have more channels, but it may also require more computation. A higher input resolution may preserve more detail, but it increases memory and processing cost.

The main idea of EfficientNet is compound scaling. Instead of scaling depth, width, and resolution independently, EfficientNet scales them together using a balanced rule:

$$d = \alpha^{\phi}, \qquad w = \beta^{\phi}, \qquad r = \gamma^{\phi},$$

where d is the network depth, w is the channel width, r is the input resolution, and ϕ controls the overall model scale. The constants α, β, and γ determine how much each dimension grows when the model is scaled up. This allows EfficientNet to build larger models in a more systematic way while maintaining a good trade-off between performance and computational cost.

Another important component of EfficientNet is the MBConv block, also known as the mobile inverted bottleneck convolution block. An MBConv block first expands the number of channels, then applies depthwise convolution, and finally projects the features back to a lower-dimensional representation. This structure is efficient because the spatial convolution is applied separately to each channel, reducing the number of parameters and operations compared with a standard convolution.

The general structure of an MBConv block can be summarized as follows:

$$X \rightarrow \text{Expansion} \rightarrow \text{Depthwise Convolution} \rightarrow \text{Squeeze-and-Excitation} \rightarrow \text{Projection}.$$

The expansion step increases the feature dimension so that the network can learn richer intermediate representations. The depthwise convolution captures local spatial patterns efficiently. The projection step reduces the number of channels again, making the block computationally compact.

EfficientNet also commonly uses squeeze-and-excitation modules. This mechanism allows the network to reweight feature channels according to their importance. In other words, the model can learn which channels should be emphasized and which channels should be suppressed. For spectrogram-based bird classification, this is useful because different channels may respond to different acoustic structures, such as short chirps, harmonic bands, frequency sweeps, or repeated syllables.

In the spectrogram-based model, the EfficientNet backbone receives an input tensor:

$$X \in \mathbb{R}^{C \times F \times T},$$

where C is the number of input channels, F is the number of mel-frequency bins, and T is the number of time frames. The backbone maps this input into a high-level feature representation:

$$H = f_{\theta}(X),$$

where f_θ denotes the EfficientNet backbone and H is the extracted feature map. Early convolutional layers capture simple local patterns in the time–frequency domain, while deeper layers combine these patterns into more abstract acoustic representations.

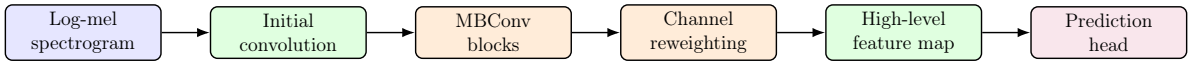


Figure 2: General EfficientNet-based spectrogram feature extraction pipeline. The log-mel spectrogram is passed through convolutional blocks to produce high-level acoustic features for the prediction head.

For bird sound classification, EfficientNet is suitable because bird vocalizations often appear as structured local patterns in spectrograms. Some species may be characterized by short high-frequency chirps, while others may have longer harmonic calls or repeated temporal patterns. EfficientNet can capture these patterns through convolutional filters at different depths. The early layers focus on local acoustic details, and the deeper layers learn more discriminative representations for species prediction.

Overall, EfficientNet provides an efficient and effective backbone for spectrogram-based audio classification. It combines balanced model scaling, efficient convolutional blocks, and channel attention mechanisms. These properties make it suitable for extracting meaningful acoustic features from log-mel spectrograms while keeping the model computationally practical.

4.3 Sound Event Detection Model

The baseline spectrogram classifier predicts species presence for an entire audio clip. Building on the SED overview above, this subsection focuses on the model formulation used to preserve temporal evidence before clip-level aggregation.

In the SED formulation, the convolutional backbone first extracts a feature map from the log-mel spectrogram. If the input spectrogram is denoted by $X \in \mathbb{R}^{1\times F\times T}$, the backbone produces a high-level representation

$$H = f_\theta(X) \in \mathbb{R}^{C\times F'\times T'},$$

where C is the number of feature channels and T' is the downsampled temporal dimension. We then aggregate over the frequency dimension to obtain a time-indexed feature sequence:

$$Z_t = \frac{1}{F'}\sum_{f=1}^{F'} H_{:,f,t}, \quad t = 1,\dots,T'.$$

This produces a sequence $Z \in \mathbb{R}^{C\times T'}$, where each time step contains information about the acoustic events occurring around that region of the clip.

Instead of producing only one global prediction directly from the entire spectrogram, the SED head produces time-wise evidence for each species. Conceptually, this allows the model to answer two related questions: which species are present in the clip, and during which time

regions is there evidence for each species. Although our final training labels are clip-level multi-label annotations, the intermediate temporal representation helps the model localize discriminative patterns.

The final clip-level prediction is obtained by aggregating the time-wise information across T' frames. This design is especially useful for bioacoustic recordings because species calls may be short, repeated, and partially overlapped with other sounds. By keeping the temporal dimension until the pooling stage, the model can focus on relevant call segments rather than treating all time frames equally.

4.4 Attention Pooling

Attention pooling is used to convert time-wise SED features into one clip-level prediction vector. A simple average over time assumes that all frames are equally important, but this assumption is weak for bird audio: many frames may contain only background noise, while only a few frames contain the target call. Attention pooling allows the model to learn which time frames should contribute more strongly to the final prediction.

Let $Z = [z_1, z_2, \ldots, z_T]$ be the sequence of temporal features produced by the SED backbone, where $z_t \in \mathbb{R}^C$. The attention module computes an attention score for each frame and each class:

$$a_t = W_a z_t + b_a.$$

The scores are normalized across time using a softmax function:

$$\alpha_{c,t} = \frac{\exp(a_{c,t})}{\sum_{k=1}^{T} \exp(a_{c,k})},$$

where $\alpha_{c,t}$ represents how much class c attends to frame t. A separate classification branch also computes frame-wise class evidence:

$$l_{c,t} = W_c z_t + b_c.$$

The clip-level logit for class c is obtained by a weighted sum over time:

$$l_c = \sum_{t=1}^{T} \alpha_{c,t} l_{c,t}.$$

Finally, a sigmoid activation converts logits into independent class probabilities:

$$p_c = \sigma(l_c).$$

This mechanism is appropriate for multi-label bird classification because different species may vocalize at different moments in the same recording. Class-specific attention allows the model to focus on different time regions for different species. For example, one species may be detected from an early chirp, while another species may appear near the end of the same clip.

Compared with global average pooling, attention pooling provides a more flexible aggregation method and helps the model handle sparse vocalizations, overlapping calls, and noisy natural recordings.

5 Training Strategy

5.1 Transfer Learning

Transfer learning is important in BirdCLEF because the number of labeled examples is highly uneven across species, while modern CNNs contain millions of parameters. Instead of training the full network from scratch, we initialize the backbone with weights pretrained on large-scale image datasets. Although natural images and spectrograms are different modalities, the early convolutional layers of pretrained image models still learn useful generic filters such as edges, textures, and local contrast patterns. In spectrograms, these filters can respond to frequency bands, harmonic contours, call onsets, and repeated acoustic motifs.

The main technical issue is the difference between spectrogram channels and RGB image channels. Most pretrained image models expect an input tensor with three channels, $\mathbf{X} \in \mathbb{R}^{3\times H\times W}$, whereas a log-mel spectrogram is naturally one-channel. There are two common solutions. The first solution is channel replication, where the same spectrogram is copied three times:

$$\mathbf{X}_{3c} = [\mathbf{X}, \mathbf{X}, \mathbf{X}].$$

This preserves compatibility with pretrained weights without modifying the first convolution. The second solution is to replace the first convolutional layer with a one-channel convolution and initialize its weights by averaging the pretrained RGB filters:

$$W_{1c} = \frac{1}{3}\left(W_R + W_G + W_B\right).$$

This approach reduces redundant input channels while still transferring the low-level filters learned from images. A third variant uses multiple acoustic channels, such as log-mel, delta, and delta-delta spectrograms, which more closely resembles a three-channel image while encoding complementary temporal information.

Fine-tuning is usually performed gradually. In the initial stage, the pretrained backbone can be partially frozen so that only the classification head and possibly the last convolutional blocks are updated. This stabilizes training and prevents large gradient updates from destroying useful pretrained representations. In later stages, more layers are unfrozen and trained with a smaller learning rate, allowing the backbone to adapt to the domain shift from natural images to bird audio spectrograms. A typical strategy uses a larger learning rate for the newly initialized classification head and a smaller learning rate for the pretrained backbone.

For the classifier head, the original single-label image classification layer is removed and replaced with a multi-label head containing K outputs. In our baseline setting, the model is optimized with binary cross-entropy, as described in the Loss Function subsection. Other

objectives such as focal-style or ranking-oriented losses may be considered in future experiments, but they are not evaluated separately in the current final result table.

In our project, transfer learning therefore serves two purposes. It accelerates convergence by reusing robust visual feature extractors, and it can support rare species by reducing the amount of species-specific data required to learn useful acoustic patterns. This makes pretrained EfficientNet backbones a natural choice for the baseline model and for later ensemble members.

5.2 Class Balancing

Based on the rare-species challenge described in Section 3.3, class balancing is treated as a training-level strategy rather than a preprocessing step. The goal is to expose the model to rare species more frequently while avoiding overfitting to the same limited recordings.

The first strategy is class-aware sampling. Instead of sampling training recordings uniformly, we increase the probability of selecting examples that contain rare species. Let n_k be the number of training examples containing species k. A simple sampling weight for an example containing class k can be defined as:

$$w_k = \frac{1}{(n_k + \epsilon)^\alpha},$$

where ϵ is a small constant for numerical stability and $\alpha \in [0, 1]$ controls the strength of rebalancing. When $\alpha = 0$, sampling is close to uniform. When α is larger, rare classes receive more emphasis. For multi-label clips, the sample weight can be computed from the maximum or average of the class weights of all species present in the clip.

However, rare-class oversampling must be used carefully. If the same few rare recordings are repeated too often, the model may memorize recording-specific background noise, microphone characteristics, or local acoustic artifacts instead of learning species-level vocalization patterns. Therefore, class-aware sampling should be combined with controlled augmentation.

Data augmentation is used to increase acoustic diversity while preserving species identity. Random cropping exposes the model to different temporal positions of the same recording. Background noise mixing makes the model less dependent on clean focal recordings. Time masking and frequency masking encourage the model to rely on robust acoustic patterns rather than a narrow time or frequency region. When waveform-level mixing is used, two audio signals are combined in the waveform domain, and the target label is computed as the element-wise maximum of the two multi-hot label vectors. This increases acoustic diversity while preserving all species that appear in either source clip.

Pseudo-labeled soundscapes can also support rare-species learning. Some rare species may appear in soundscape-style recordings even if they are underrepresented in the labeled directed recordings. However, pseudo-labels must be used conservatively because low-confidence predictions can amplify confirmation bias, especially for rare classes. For this reason, pseudo-label usage should be combined with confidence thresholding and careful sample selection.

Loss-level reweighting, focal-style objectives, and label smoothing may also be considered as optional strategies for handling imbalance and noisy labels. However, since these components

are not evaluated separately in the final result table, this report does not claim their individual contribution. In our analysis, class balancing is mainly discussed as a combination of class-aware sampling, controlled augmentation, and conservative pseudo-label usage.

5.3 Loss Function

In this project, bird sound classification is formulated as a multi-label classification problem. A single audio segment may contain more than one species, so the model should not use a softmax output, which assumes that only one class is correct. Instead, the model produces one independent logit for each species, and each logit is converted into a probability using the sigmoid function:

$$\hat{y}_{i,c} = \sigma(z_{i,c}) = \frac{1}{1 + \exp(-z_{i,c})},$$

where $z_{i,c}$ is the logit for class c in sample i, and $\hat{y}_{i,c}$ is the predicted probability that the species is present.

The corresponding loss function is binary cross-entropy (BCE). BCE treats each class as an independent binary prediction problem. For a mini-batch of N samples and C species, the multi-label BCE loss is:

$$\mathcal{L}_{BCE} = -\frac{1}{NC} \sum_{i=1}^{N} \sum_{c=1}^{C} \left[y_{i,c} \log(\hat{y}_{i,c}) + (1 - y_{i,c}) \log(1 - \hat{y}_{i,c}) \right],$$

where $y_{i,c} \in \{0, 1\}$ indicates whether class c is present in sample i. If the species is present, the loss encourages the predicted probability to be close to 1. If the species is absent, the loss encourages the predicted probability to be close to 0.

In implementation, BCE is often computed directly from logits using binary cross-entropy with logits. This combines the sigmoid operation and the BCE loss into one numerically stable function. Overall, BCE is suitable for multi-label bird sound classification because it allows each species to be predicted independently, making it possible for the model to detect multiple species in the same audio segment.

6 Semi-Supervised and Teacher-Student Approaches

6.1 Overview

This section studies three related teacher-based approaches: pseudo-labeling, Noisy Student training, and self-distillation. Pseudo-labeling uses teacher predictions to exploit unlabeled soundscapes, Noisy Student trains a student model with stronger noise and augmentation, and self-distillation transfers teacher outputs to a student model through soft targets.

6.2 Pseudo Labeling

6.2.1 Motivation

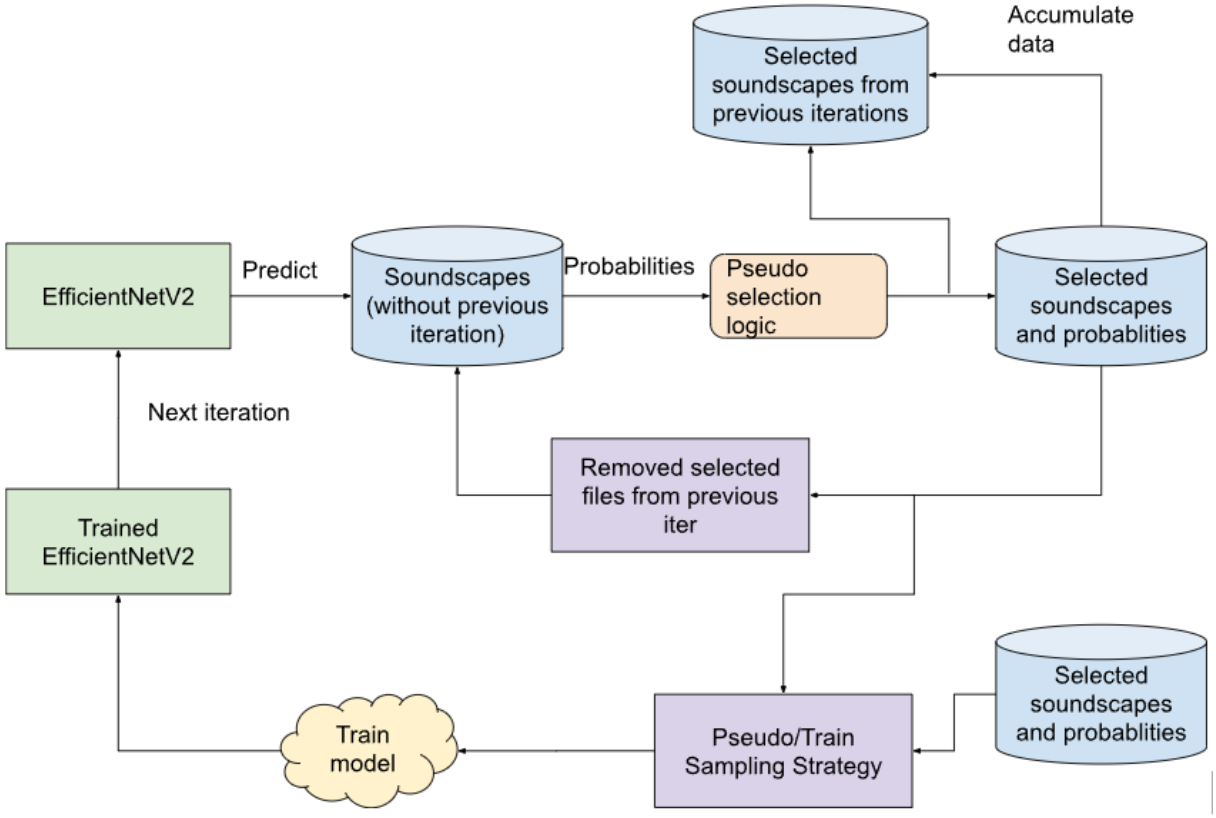
As mentioned above, pseudo-labeling is motivated by two main characteristics of the BirdCLEF 2025 dataset: limited labeled data for many species and the availability of long unlabeled soundscape recordings. Although the training set contains 28,564 labeled audio samples, the distribution across species is highly imbalanced. Some species have hundreds of recordings, while 39 species have fewer than 10 samples and the rarest species has only 2 samples. This makes it difficult for a purely supervised model to learn reliable acoustic patterns for rarer classes.

At the same time, our dataset provides soundscape-style recordings that are closer to the test environment. These recordings are collected through passive acoustic monitoring and may contain bird calls, animal sounds, and environmental noise. Even when they are unlabeled, they still contain useful information about the acoustic domain of the test set.

Pseudo-labeling provides a practical way to use this unlabeled data without requiring expensive manual labeling. The idea is to first train a teacher model on the labeled recordings, then use that model to predict species probabilities on unlabeled soundscape chunks. Predictions with sufficiently high confidence are treated as additional training targets. In this way, the model can gradually expand its training set and learn from examples that better match the passive-acoustic-monitoring test distribution.

This strategy is especially useful for reducing domain shift. Directed recordings from sources such as Xeno-Canto often focus on a target species and may contain clearer vocalizations, while soundscapes are longer, noisier, and more likely to contain overlapping species. By adding pseudo-labeled soundscape chunks to training, the model is exposed to background noise, overlapping calls, and realistic recording conditions that are not fully represented by the original labeled data.

6.2.2 Pseudo-Label Generation



The first step in our pseudo-labeling pipeline is to load the soundscape file into a dataset. There are a total of 9726 soundscape files, totaling around 162 hours of audio recordings. We split these into five-second chunks using the audio segmentation strategy described earlier.

Next, we use our best model to run inference on these soundscape data and save the predictions to use in the next step. The output of each soundscape audio is the probability vector of the classes. The shape of the result soundscape predictions is (206,9726).

The probabilities then go into the pseudo-selection logic. For each five-second chunk, we compute the maximum predicted probability. Chunks whose maximum predicted probability is less than 0.5 are discarded. For kept chunks, all class probabilities below 0.1 are set to zero. The remaining probability vector is stored as a soft target rather than a hard binary label for knowledge distillation.

In the next step, we accumulate the predicted pseudo-labels from previous iterations to create a new pseudo-labeled dataset. In our code, running this process keeps 39172 chunks out of 116172 chunks, a selection ratio of 33.6% from soundscape files.

Next, we apply our sampling strategy to use pseudo-labels along with real labels. Since the original label is more trustworthy than the pseudo-label, the sampling ratio for labeled data should be higher than the pseudo-labeled data. In code, the ratio is set to 60/40: if a class appears in the pseudo-labeled dataset, it has a 40% chance to use the pseudo-label source.

Finally, we train with the new pseudo-labels. In code, we choose the number of pseudo epochs to be 3. After training, we observe a decrease in training loss and an increase in internal AUC. Detailed results are reported in Table 2.

6.2.3 Confidence Thresholding

Pseudo-labeling can increase the amount of training data by assigning labels to unlabeled audio segments. However, pseudo-labels are generated by a model, so they are not always correct. If all predicted labels are used directly, noisy pseudo-labels may mislead the student model and reduce internal learning quality. Confidence thresholding is therefore used as a filtering strategy to control pseudo-label quality.

Let the teacher model output a probability vector for an unlabeled audio segment:

$$\mathbf{p}_i = [p_{i,1}, p_{i,2}, \ldots, p_{i,C}],$$

where $p_{i,c}$ is the predicted probability that class c is present in sample i, and C is the number of target classes. A simple confidence thresholding rule keeps a pseudo-label only when the predicted probability is high enough:

$$\hat{y}_{i,c} = \begin{cases} 1, & \text{if } p_{i,c} \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

Here, τ is the confidence threshold. A high threshold keeps only very confident predictions, while a low threshold keeps more pseudo-labels but may introduce more noise.

In multi-label bird sound classification, thresholding is more complicated than in single-label classification. A sound segment may contain multiple species, so each class should be considered independently. Instead of selecting only the class with the highest probability, the model may keep all species whose probabilities exceed the threshold. This is important because real soundscapes often contain overlapping vocalizations from different species.

There are two common ways to choose thresholds. The first option is to use a global threshold shared by all classes:

$$\tau_1 = \tau_2 = \cdots = \tau_C = \tau.$$

This approach is simple and easy to implement, but it may not work equally well for all species. Common species may receive more confident predictions, while rare or difficult species may have lower predicted probabilities even when they are present.

The second option is to use class-wise thresholds:

$$\hat{y}_{i,c} = \begin{cases} 1, & \text{if } p_{i,c} \geq \tau_c, \\ 0, & \text{otherwise,} \end{cases}$$

where τ_c is a separate threshold for class c. This approach can better handle class imbalance, but it requires careful validation. If the threshold for a rare species is too high, the model may discard most useful pseudo-labels for that species. If it is too low, many false positives may be introduced.

The choice of threshold creates a trade-off between pseudo-label precision and pseudo-label

coverage. A higher threshold usually increases precision because the selected pseudo-labels are more reliable. However, it also reduces coverage because fewer unlabeled samples are used for training. A lower threshold increases coverage but may reduce label quality. Therefore, thresholds should be selected based on validation performance rather than chosen arbitrarily.

In practice, confidence thresholding should be treated as a noise-control mechanism. It does not guarantee that all selected pseudo-labels are correct, but it reduces the probability of training on highly uncertain predictions. For bird sound classification, this is especially useful because the data may contain background noise, overlapping species, and weak vocalizations. A well-chosen threshold helps the model benefit from unlabeled soundscapes while limiting the negative effect of incorrect pseudo-labels.

6.2.4 Soft Labels

Pseudo-labels do not always have to be converted into hard binary labels. Instead of assigning each class a target value of exactly 0 or 1, the model can use the teacher's probability outputs directly as soft labels. This is useful because the probability vector contains more information than a hard label.

For an unlabeled sample x_i, suppose the teacher model outputs:

$$\mathbf{q}_i = [q_{i,1}, q_{i,2}, \ldots, q_{i,C}],$$

where $q_{i,c} \in [0, 1]$ represents the teacher's confidence that class c is present. A hard pseudo-label would convert this vector into binary values using a threshold:

$$q_{i,c} \rightarrow \hat{y}_{i,c} \in \{0, 1\}.$$

In contrast, a soft-label approach keeps the original probability value $q_{i,c}$ as the training target.

The main advantage of soft labels is that they preserve uncertainty. For example, if the teacher predicts a probability of 0.55 for one species and 0.95 for another species, both predictions may become positive labels after thresholding. However, they do not have the same confidence. The 0.95 prediction is much more reliable than the 0.55 prediction. Soft labels preserve this difference, while hard labels remove it.

Soft labels are also helpful when species are acoustically similar. Some bird calls may share similar frequency patterns or temporal structures. In such cases, the teacher may assign moderate probabilities to several related classes. A hard label forces the student to treat one class as fully present and the others as absent. A soft label allows the student to learn a smoother target distribution, which can reduce overconfidence.

When soft labels are used, the training objective can still be based on binary cross-entropy. The difference is that the target is no longer restricted to 0 or 1. For a student prediction $\hat{y}_{i,c}$

and a soft target $q_{i,c}$, the soft-label BCE loss is:

$$\mathcal{L}_{soft} = -\frac{1}{NC}\sum_{i=1}^{N}\sum_{c=1}^{C}\left[q_{i,c}\log(\hat{y}_{i,c}) + (1-q_{i,c})\log(1-\hat{y}_{i,c})\right].$$

This formulation allows the student model to learn from the teacher's confidence values directly.

In multi-label audio classification, soft labels are more appropriate than one-hot labels. A one-hot label assumes that only one class is correct, while bird sound recordings may contain multiple species at the same time. Therefore, the correct representation is a multi-label target vector, where each class has its own target value. This target can be either a hard binary value or a soft probability value.

Overall, soft labels provide a more informative form of supervision for pseudo-labeled data. They reduce the loss of information caused by hard thresholding and allow the student model to learn from uncertain predictions more smoothly. However, soft labels still depend on the quality of the teacher model. If the teacher is poorly calibrated or consistently wrong, the soft targets may also be misleading. For this reason, soft labels are often combined with confidence thresholding, validation monitoring, and careful selection of pseudo-labeled samples.

6.3 Noisy Student Training

Noisy Student is a teacher–student training approach that builds on the pseudo-label representation described in the Soft Labels subsection. A teacher model is first trained on labeled recordings and then used to generate pseudo-labels for unlabeled soundscape segments. The student model is trained using both ground-truth labels and teacher-generated pseudo-labels.

The unique contribution of Noisy Student is not the soft-label representation itself, but the way the student is trained with stronger noise and augmentation than the teacher. These perturbations can include random cropping, input noise, masking, MixUp, or other data augmentation techniques. The purpose is to force the student to learn stable acoustic patterns that remain correct under different input variations. As a result, the student may show stronger internal learning behavior than the original teacher, although this should be verified on held-out soundscape data.

Formally, let (x_l, y_l) be a labeled example and let x_u be an unlabeled example. The teacher model T predicts a pseudo-label for the unlabeled example:

$$\hat{y}_u = T(x_u).$$

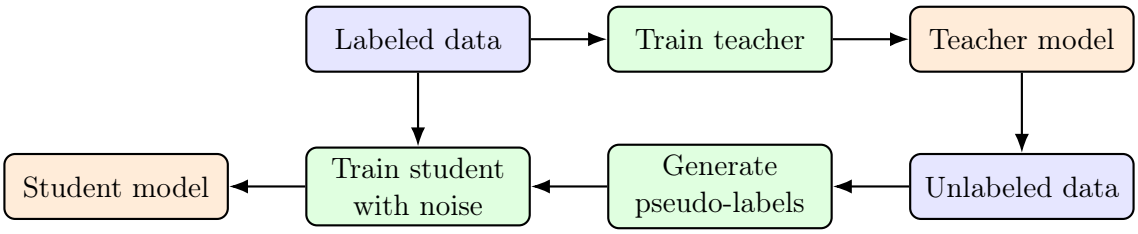
The student model S is then trained on both labeled and pseudo-labeled data:

$$\mathcal{L} = \mathcal{L}_{sup}(S(x_l), y_l) + \lambda_u \mathcal{L}_{unsup}(S(\tilde{x}_u), \hat{y}_u),$$

where $\tilde{x}_u$ denotes an augmented or noisy version of the unlabeled input, and λ_u controls the contribution of the pseudo-labeled data. In multi-label classification, binary cross-entropy is

commonly used for both terms.

Noisy Student can also be applied iteratively. After the student has been trained, it can be used as a new teacher to generate updated pseudo-labels for the unlabeled data. A new student is then trained again using these updated pseudo-labels. This creates a self-training loop in which the quality of the pseudo-labels and the robustness of the model may gradually improve.



The main advantage of Noisy Student is that it can exploit unlabeled data without requiring manual annotation. It also improves robustness because the student is trained under stronger perturbations. However, the method must be used carefully. If the teacher produces many incorrect pseudo-labels, the student may learn and reinforce these errors. Therefore, pseudo-label quality, augmentation strength, and validation performance should be monitored during training.

6.4 Self Distillation

Self-distillation is a teacher–student training strategy in which knowledge learned by a trained model is reused to supervise another model. Unlike the Noisy Student discussion, this subsection focuses on the training objective: the student learns from both the original hard labels and the teacher soft targets defined earlier in the Soft Labels subsection.

The motivation is that ground-truth labels are often incomplete or noisy in bioacoustic data. A recording may be labeled with one primary species even though other species are faintly present in the background. Teacher probabilities can provide smoother supervision than strict binary targets, while the original labels still anchor the student to verified annotations.

Let y be the original multi-label target and let q be the teacher's predicted probability vector. The student produces logits s and probabilities $p = \sigma(s)$. We train the student with a combination of hard-label loss and soft-label distillation loss:

$$\mathcal{L}_{\text{student}} = \lambda \mathcal{L}_{\text{BCE}}(s, y) + (1 - \lambda)\mathcal{L}_{\text{BCE}}(s, q),$$

where λ controls the balance between the original labels and the teacher's soft targets. The first term ensures that the student still respects the verified labels, while the second term transfers additional knowledge from the teacher.

In practice, self-distillation is useful only when the teacher model is sufficiently reliable. If the teacher is weak, its soft targets may introduce noise and reinforce wrong predictions. Therefore, the teacher should first be validated carefully using internal metrics such as macro AUC

and mean average precision. Low-confidence predictions can also be filtered or down-weighted to reduce confirmation bias. When applied properly, self-distillation smooths supervision and transfers teacher knowledge to the student; any generalization benefit should be verified on held-out soundscape data.

7 Inference Strategy and Post-Processing

Inference is an important part of our pipeline because the test recordings are long soundscapes rather than isolated short clips. A single bird call may occur only briefly, and different species may appear at different moments in the same one-minute recording. Therefore, the inference stage should preserve temporal resolution and should not depend on only one model or one crop.

7.1 Multi-Model Ensemble

During inference, each audio segment is converted into a log-mel spectrogram using the same preprocessing parameters as in training. A trained model outputs a probability vector for all target classes:

$$p^{(m)} = [p_1^{(m)}, p_2^{(m)}, \ldots, p_C^{(m)}],$$

where m denotes the model index and C is the number of target classes. The simplest ensemble combines M models by arithmetic averaging:

$$\bar{p}_c = \frac{1}{M} \sum_{m=1}^{M} p_c^{(m)}.$$

This averaging reduces the variance of individual models. Diversity can come from different folds, checkpoints, random seeds, pseudo-label versions, training configurations, or window alignments within compatible EfficientNet-based models. For example, one checkpoint may detect high-frequency calls better, while another may be more stable on noisy soundscape segments. Averaging their probability outputs can therefore improve prediction stability without changing the training labels.

If validation results show that some models are consistently stronger, weighted averaging can be used:

$$\bar{p}_c = \sum_{m=1}^{M} w_m p_c^{(m)}, \quad \sum_{m=1}^{M} w_m = 1.$$

However, in a course project setting, simple averaging is easier to justify and less likely to overfit the validation split. Therefore, our main inference strategy is based on probability averaging across compatible EfficientNet-based checkpoints, folds, and window alignments.

7.2 Regular and Shifted-Window Inference

For soundscape recordings, we divide each one-minute audio file into consecutive five-second chunks. This produces 12 regular windows: 0–5s, 5–10s, 10–15s, and so on until 55–60s. This matches the prediction format and preserves a fixed temporal resolution.

However, a vocalization may occur near the boundary between two neighboring windows. If only non-overlapping windows are used, part of the call may be split across chunks and become harder to detect. To reduce this boundary problem, we also use shifted windows with a 2.5-second offset, such as 2.5–7.5s, 7.5–12.5s, and so on. The regular-window predictions and shifted-window predictions are then blended.

For a middle chunk, the final probability can be computed as

$$p_t^{\text{blend}} = 0.50p_t^{\text{regular}} + 0.25p_{t-1}^{\text{shifted}} + 0.25p_t^{\text{shifted}}.$$

This combines the prediction from the current regular segment with the two neighboring shifted segments that overlap it. The strategy is useful because it allows the classifier to see the same acoustic event under slightly different temporal alignments.

7.3 Temporal Smoothing

Predictions over time should be locally consistent because natural soundscapes are continuous. If a species is detected strongly in one chunk, it may also have some evidence in adjacent chunks. To reduce unstable frame-to-frame jumps, we apply a simple temporal smoothing rule:

$$p_t^{\text{smooth}} = 0.10p_{t-1}^{\text{blend}} + 0.80p_t^{\text{blend}} + 0.10p_{t+1}^{\text{blend}}.$$

For the first and last chunks, only the available neighboring chunk is used. This smoothing step is lightweight and does not require retraining. It reduces isolated false spikes and improves the temporal stability of the final prediction sequence.

7.4 Probability Adjustment and Risks

Probability post-processing can also be used to adjust the confidence distribution. One simple method is power adjustment:

$$\hat{p}_c = p_c^{\gamma},$$

where γ is a positive exponent. If $\gamma > 1$, weak predictions are suppressed and the model becomes more conservative. If $0 < \gamma < 1$, predictions become softer. This technique can be useful when a model produces many weak false positives, but the exponent should be selected using validation folds to avoid overfitting.

The main advantage of the complete inference strategy is robustness. Model ensembling reduces model-specific errors, shifted windows reduce boundary effects, and temporal smoothing stabilizes predictions across neighboring chunks. The main risk is computational cost: using

many models and multiple window alignments increases inference time. Therefore, in practical deployment, the ensemble size should be chosen based on the trade-off between performance and runtime.

8 Experimental Results

This section presents the internal experimental results of our bird sound classification methods. The purpose is to compare the relative contribution of the main training and inference strategies implemented in our project. To keep the comparison consistent, all methods are based on the same EfficientNet backbone family and use the same spectrogram-based preprocessing pipeline whenever possible.

Since we do not have a separate held-out test set in this internal experiment, the reported values are based on training-set evaluation. Therefore, the scores should be interpreted as *in-sample* performance. These results are useful for checking whether each method can learn the training data and for comparing different components under the same setting. However, they should not be interpreted as true generalization performance on unseen data.

8.1 Evaluation Metric

We evaluate the methods using ROC-AUC. In multi-label bird sound classification, each audio segment can contain multiple species, so the model outputs one probability for each class:

$$\hat{\mathbf{y}}_i = [\hat{y}_{i,1}, \hat{y}_{i,2}, \ldots, \hat{y}_{i,C}],$$

where C is the number of target classes and $\hat{y}_{i,c}$ is the predicted probability that class c is present in sample i.

For each class, ROC-AUC measures how well the model ranks positive samples above negative samples. A higher ROC-AUC means that the model gives higher scores to true positive examples more consistently. For the multi-label setting, we compute AUC for each valid class and then average the results:

$$\text{MacroAUC} = \frac{1}{|\mathcal{C}_{valid}|} \sum_{c \in \mathcal{C}_{valid}} \text{AUC}_c.$$

Here, $\mathcal{C}_{valid}$ is the set of classes that contain both positive and negative samples in the evaluation data.

In this section, we report **Train ROC-AUC** only. This metric is used to compare the internal behavior of different methods under the same evaluation condition. Since the evaluation is performed on training data, the reported values may be optimistic and should be interpreted carefully.

8.2 Ablation Study

The ablation study compares the main training methods implemented in our project. All methods use EfficientNet as the shared backbone, so the difference between rows mainly reflects the effect of the added training strategy rather than a change in backbone architecture. The compared methods include the supervised EfficientNet baseline, pseudo-labeling, Noisy Student training, and self-distillation.

Table 2: Internal ablation study using Train ROC-AUC. All methods use EfficientNet as the shared backbone.

ID	Method	Main idea	Train ROC-AUC
0	Ensemble EfficientNet	Supervised spectrogram-based baseline using labeled data only	0.9339
1	EfficientNet + Pseudo Labeling	Add pseudo-labeled unlabeled audio to provide extra supervision	0.9559
2	EfficientNet + Noisy Student	Train a student model using pseudo-labels and stronger augmentation	0.9607
3	EfficientNet + Self Distillation	Use teacher probability outputs as soft targets for student training	0.9560

The supervised EfficientNet setting is used as the baseline and obtains a Train ROC-AUC of 0.9339. After adding pseudo-labeling, the score increases to 0.9559, which suggests that pseudo-labeled audio can provide useful additional supervision. Noisy Student achieves the highest Train ROC-AUC among the training methods, reaching 0.9607. This suggests that combining pseudo-labels with stronger student augmentation gives the strongest in-sample learning behavior in this experiment. Self-distillation also improves internal Train ROC-AUC over the baseline and reaches 0.9560, suggesting that soft teacher predictions can provide useful training signals under this in-sample evaluation setting.

Among these methods, Noisy Student gives the best internal result. Compared with the baseline, it improves Train ROC-AUC by:

$$0.9607 - 0.9339 = 0.0268.$$

This improvement indicates that teacher–student learning is beneficial under our internal evaluation setting. However, because the result is measured on training data, this improvement should be interpreted as evidence of better fitting and stronger internal learning behavior, not as a guaranteed improvement on unseen data.

8.3 Inference and Post-Processing Result

Besides training strategies, we also evaluate the inference and post-processing pipeline. In this part, the model predictions are generated as sigmoid probabilities for all classes. When multiple predictions are available from different checkpoints, folds, or windows, they can be aggregated into a final probability vector. Post-processing can then adjust the final probabilities based on the selected inference strategy.

Table 3: Internal result of the inference and post-processing pipeline using Train ROC-AUC.

ID	Pipeline setting	Description	Train ROC-AUC
0	Single EfficientNet prediction	Use one trained model to produce class probabilities	0.9023
1	Checkpoint / fold aggregation	Combine probability outputs from multiple checkpoints or folds	0.9231
2	Window-based aggregation	Aggregate predictions from multiple audio windows or segments	0.9271
3	Final inference and post-processing pipeline	Apply the selected aggregation and probability adjustment strategy	0.9493

The inference and post-processing results show that prediction aggregation improves the internal ROC-AUC compared with using a single EfficientNet prediction. The single-model inference setting obtains 0.9023, while checkpoint or fold aggregation improves the score to 0.9231. Window-based aggregation further improves the score to 0.9271, suggesting that using multiple audio windows helps capture more complete evidence from the recording.

The final inference and post-processing pipeline achieves the best score in this group, with a Train ROC-AUC of 0.9493. Compared with the single prediction setting, the improvement is:

$$0.9493 - 0.9023 = 0.0470.$$

This shows that inference-time aggregation and post-processing have a clear effect on the internal final probability outputs. The improvement is especially important for audio data because bird vocalizations may appear only in short regions, and a single window may miss useful acoustic evidence.

8.4 Final Result Summary

Based on the internal evaluation, the strongest training strategy and the strongest inference setting are summarized separately. This separation is important because the two results come from different internal comparisons. The training-method ablation compares how different learning

strategies affect Train ROC-AUC, while the inference and post-processing table compares how prediction aggregation affects the final probability outputs.

Table 4: Final internal result summary using Train ROC-AUC. The training strategy and inference pipeline are reported separately because they are evaluated in different internal comparisons.

Component	**Train ROC-AUC**	**Interpretation**
Best training strategy	0.9607	EfficientNet + Noisy Student gives the strongest in-sample learning behavior among the compared training methods.
Best inference / post-processing setting	0.9493	The final aggregation and probability adjustment pipeline gives the strongest result among the compared inference settings.

Overall, the internal results suggest that both training strategy and inference-time aggregation are important. Noisy Student gives the highest Train ROC-AUC in the training-method comparison, suggesting that pseudo-label-based student training with stronger augmentation is effective under the in-sample evaluation setting. Separately, the final inference and post-processing pipeline gives the highest Train ROC-AUC among the inference settings, showing that aggregation and probability adjustment improve prediction stability.

Since all reported scores are based on Train ROC-AUC, these values should be interpreted as internal comparison results rather than true generalization performance. A held-out soundscape evaluation would be needed to confirm whether the same ranking holds on unseen data.

9 Discussion

The internal experiments suggest that bird sound recognition benefits from combining several modeling ideas rather than relying on one isolated technique. Spectrogram-based CNNs provide a strong starting point because many bird calls have clear time-frequency structures. EfficientNet is a practical backbone because it can reuse pretrained visual features while remaining computationally efficient.

The model design suggests that temporal modeling is important for this task, although its isolated contribution is not directly measured in the final ablation table. A clip-level classifier treats the whole spectrogram as one image, but many bird calls occur only in short regions. Sound event detection and attention pooling are designed to help the model focus on short and sparse vocalizations, but a separate SED-only ablation would be needed to quantify their individual contribution.

The second finding is that class imbalance must be handled explicitly. The dataset contains many rare species, and a naive training strategy can bias the model toward common classes. Class-aware sampling, optional class weighting or focal-style objectives, and controlled augmentation are designed to help rare-class learning, but their individual contribution is not isolated in our final result table. Rare species remain difficult because some classes have very few labeled recordings, and aggressive oversampling may cause overfitting.

The third finding is that unlabeled or weakly labeled soundscape data can be useful when used carefully. Pseudo-labeling exposes the model to acoustic conditions closer to the test domain, including background noise and overlapping calls. However, pseudo-labeling also creates the risk of confirmation bias. Confidence thresholding and soft labels reduce this risk by filtering uncertain predictions and preserving teacher confidence.

Another important reason why augmentation-based training is useful is the domain shift between labeled directed recordings and soundscape-style audio. Labeled recordings often contain cleaner and more isolated target vocalizations, while soundscape-style recordings contain background noise, silence, overlapping species, and environmental interference. Therefore, augmentations such as random cropping, time masking, frequency masking, background noise mixing, and MixUp expose the student model to less clean and more diverse acoustic conditions. This is consistent with the Noisy Student result: the student is trained on pseudo-labeled soundscapes under stronger perturbations, which makes it less dependent on clean directed recordings and improves its in-sample learning behavior. However, because the reported results are based on Train ROC-AUC, this robustness should still be verified on held-out soundscape data.

Finally, ensemble inference suggests potential robustness. Different folds or model variants may make different errors, so averaging their probabilities produces more stable predictions. Shifted-window inference and temporal smoothing are designed to reduce boundary effects and unstable chunk-level spikes, but their contribution should be verified on held-out soundscape data. The main limitation of this strategy is inference cost, since evaluating many models and multiple windows increases runtime.

Overall, the main limitation of our project is that the reported in-sample Train ROC-AUC may not fully reflect performance on unseen soundscapes. Domain shift, incomplete labels, rare species, and noisy backgrounds remain challenging. Future work should include stronger validation protocols, class-wise threshold calibration, more systematic pseudo-label filtering, and more efficient ensemble compression for deployment.

10 Conclusion

This project studied bird and wildlife sound recognition using the BirdCLEF dataset and a spectrogram-centered machine learning pipeline. We formulated the task as multi-label classification and investigated several complementary methods: spectrogram-based CNN classification, EfficientNet backbones, sound event detection, attention pooling, class imbalance handling, pseudo-labeling, Noisy Student training, self-distillation, and ensemble inference.

The internal results suggest that a strong bioacoustic recognition system requires both a good acoustic representation and a robust training strategy. Log-mel spectrograms make bird calls visible as time-frequency patterns, EfficientNet-based models extract discriminative acoustic features, SED and attention pooling are intended to support temporal localization, and BCE-based losses provide a basic objective for multi-label prediction. Semi-supervised learning and teacher-student methods further exploit unlabeled soundscapes, while ensemble inference suggests improved prediction stability.

In conclusion, the project demonstrates how modern deep learning methods can be adapted from computer vision and semi-supervised learning to practical bioacoustic monitoring. Although rare classes, noisy labels, and domain shift remain difficult, the combination of spectrogram representation, temporal modeling, imbalance-aware training, and ensemble inference provides a strong foundation for future bird sound recognition systems.